

MÉTODOS COMPUTACIONALES 2: TAREA 2

Transformada de Fourier y análisis de señales

Por Santiago Henao Castellanos

Puede usar cualquier librería para el análisis y la FFT, pero debe usar su propia implementación para la transformada no uniforme.

No olvide consultar los “términos y condiciones” para la entrega de trabajos, en Contenido en bloque neón.

Esta tarea vale el 7% de la nota, o sea 70pt. La nota máxima es de 73pt.

Para que este ejercicio se atienda en cuenta, debe implementar y usar una función llamada

```
Fourier_transform(x::np.ndarray, y::np.ndarray, k::float) → complex
```

que evalúe la transformada de Fourier general (no uniforme) en una sola frecuencia:

$$\mathcal{F}\{y(x)\}(k) = \int_{\mathbb{R}} e^{-2\pi i k x} y(x) \, dx \approx \sum_{i=0}^{N-1} y_i e^{-2j \pi k x_i}$$

Aquí x es posición y k es el número de onda lineal $k = \frac{1}{\lambda}$. También podrían ser tiempo t y frecuencia lineal f .

Puede usar la convención de signo o normalización que desee, sus resultados no deberían cambiar.

La entrega de este ejercicio no puede contener comentarios ni strings descriptivos. Se calificará globalmente como cero de lo contrario.

1. Principio de indeterminación de las ondas (10pt)

Obtenga la transformada de Fourier de las funciones ventana tipo caja y tipo Gauss

$$\text{Box}_a(x) = \begin{cases} 1 & \text{if } |x| < \frac{a}{2} \\ 0 & \text{otherwise} \end{cases} \quad \text{Gauss}_\sigma(x) = e^{-\frac{1}{2}\left(\frac{x}{\sigma}\right)^2}$$

Halle el FWHM de la transformada (Δk , eje y) como función del FWHM de la ventana (Δx , eje x), para varios valores de a o de σ según sea el caso.

Debe guardar una gráfica en PDF de sus FWHM (como scatter), junto con un ajuste funcional de su preferencia (como plot). Debe incluir en la gráfica la fórmula usada y los valores de los parámetros encontrados.

5 puntos por cada gráfica: `1-box.pdf` y `1-gauss.pdf`.

Pista: Recuerde que para una Gaussiana $\text{FWHM} = 2\sigma\sqrt{2\ln 2}$.

Pista: use escala log-log. Recuerde que una línea en log-log es $e^m x^b$ en escala normal. Use un número apropiado de cifras significativas.

Para pensar: ¿qué pasa con $\Delta x \Delta k$?

2. Análisis de señales (30pt)

2.A. Sismología estelar, análisis multiarmónico

Los datos `star.npy` consisten en dos arrays: el tiempo en el que se tomó la medida (en días) y otro que mide el brillo de una estrella en el bulbo galáctico. Esta estrella se encuentra pulsando pseudo-periódicamente; dos de sus modos radiales se encuentran excitados al mismo tiempo. Al ser un sistema tan complicado, la frecuencia del primer armónico f_1 no es una fracción simple de la frecuencia fundamental f_0 .

El procedimiento para análisis multiarmónico es el siguiente:

- Use la transformada de Fourier para hallar el espectro de los datos.
- El máximo pico del espectro será la frecuencia que esté presente en mayor medida.
- Ajuste una serie de Fourier con esa frecuencia.
 - Para esto se recomienda pasar los datos $\{t_i, y_i\}$ a diagrama de fase $\left\{\text{mod}\left(t_i, \frac{1}{f}\right), y_i\right\}$, para visualizar el ajuste.
- Reste la serie de Fourier obtenida a los datos iniciales: `Y_new = Y_old - f(t)`
- Repita hasta que no hayan picos significativos en el espectro.

Cada ciclo de este procedimiento extraerá las frecuencias de los modos independientes de los datos. Cada vez debe guardar una serie de Fourier que modele ese modo, y al final debería tener un modelo completo $f_0(t) + f_1(t) + \dots$.

(10pt, 2.A.harmonics.png) Para el modo fundamental $f_0(t)$ y el primer armónico $f_1(t)$, realice un diagrama de fase con los datos aislados; es decir, grafique el modelo $f_0(t)$ junto con los datos $y(t_i) - f_1(t_i)$, y en otro subplot el modelo $f_1(t)$ junto con los datos $y(t_i) - f_0(t_i)$.

(5pt, 2.A.fit.pdf) Grafique los datos originales como scatter y el modelo completo, para ver cómo ajustan. Sólo grafique el tiempo desde el día 104 hasta el 112.

Datos cortesía de [OGLE](#).

Pista: los datos no están uniformemente muestreados. Hay huecos de approx. 1 día en los datos. 1/día no es una frecuencia válida, debe saber cómo removerla.

Pista: Use un paso en frecuencia menor que $1/\Delta t$, donde Δt es la duración temporal total de los datos.

Pista: las frecuencias principales pueden estar desde frecuencia 0 hasta frecuencia 4/día.

2.B. Cambio climático, modelo predictivo

En `city_weather.parquet` encontrará datos de la temperatura a las 5:00 am para diversas ciudades, diariamente, desde 1980 hasta 2024. En `city_coordinates.csv` encontrará la latitud y longitud de las ciudades. (Puede leer el formato parquet con pandas, es muy usado en ciencia de datos).

(5pt) Obtenga la media móvil anual (ancho de 365 días) usando un filtro ideal en Fourier, y una regresión lineal general para cada ciudad. Grafique para unas pocas ciudades de su preferencia (menos de 5) esta media móvil y la línea de tendencia, guarde como `2.B.tendencia.pdf`.

Reste la línea de tendencia de los datos antes de proceder.

(5pt) Use la transformada de Fourier (puede usar FFT) para hallar la frecuencia de oscilación de la temperatura de cada ciudad. Ajuste una serie de Fourier con múltiplos enteros de esa frecuencia, con pocos términos. Grafique la amplitud principal de la serie para cada ciudad como función de la latitud en `2.B.amplitudes.pdf`.

(5pt) Prediga las temperaturas de todas las ciudades en la fecha de la entrega de la tarea, usando su ajuste de línea de tendencia + serie de Fourier. Bono de 3pt si para su ajuste es capaz de hacer que las fechas más recientes pesen más. Guarde sus predicciones en `2.B.city_predictions.csv`, indicando la ciudad y la predicción en °C.

Datos cortesía de [Wolfram Research Inc.](#)

Pista: los datos están uniformemente muestreados, pero tienen huecos. Cuidado.

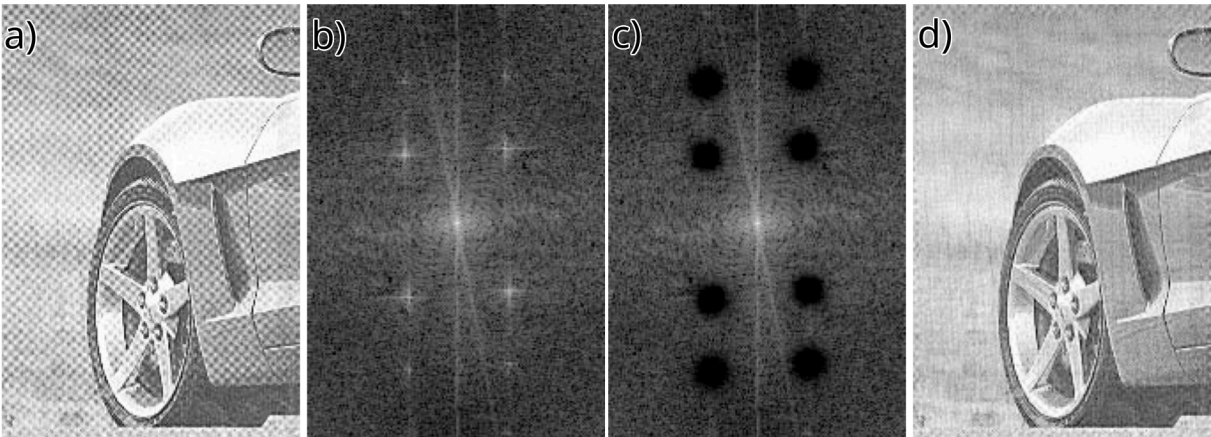
Pista: Los datos de Antigua son de dudosa calidad.

3. Análisis de imágenes (30pt)

3.A. Ruido periódico

Una imagen se compone realmente de tres “canales” de colores rojo, verde y azul. Encontrará en el archivo `images.npz` muchos arrays que contienen imágenes pequeñas infectadas con ruido periódico.

Para las 5 imágenes que más le gusten, elimine el ruido periódico de cada canal RGB tratando de recuperar la imagen original, de la siguiente manera:



(a: imagen con ruido periódico, b: transformada de fourier con `fftshift`, c: transformada con los picos exteriores en cero, d: transformada inversa de c), ahora si nruido periódico. Note que las escalas de la transformada son logarítmicas.)

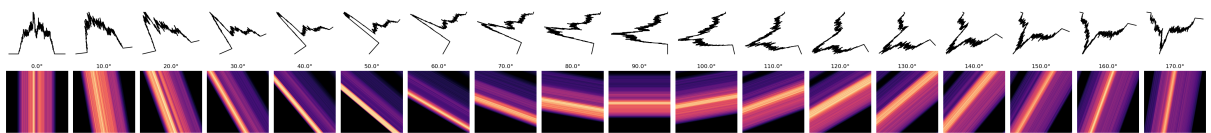
(3pt por imagen) Usando `matplotlib`, guarde su top 5 de imágenes resultantes como `3.A.1.png`, `3.A.2.png`, ..., `3.A.5.png`. Cada png debe tener la imagen antes, la transformada antes, la transformada después, y la imagen después de su filtro, como se mostró arriba. La transformada puede ser de cualquier canal.

Datos de... por ahí.

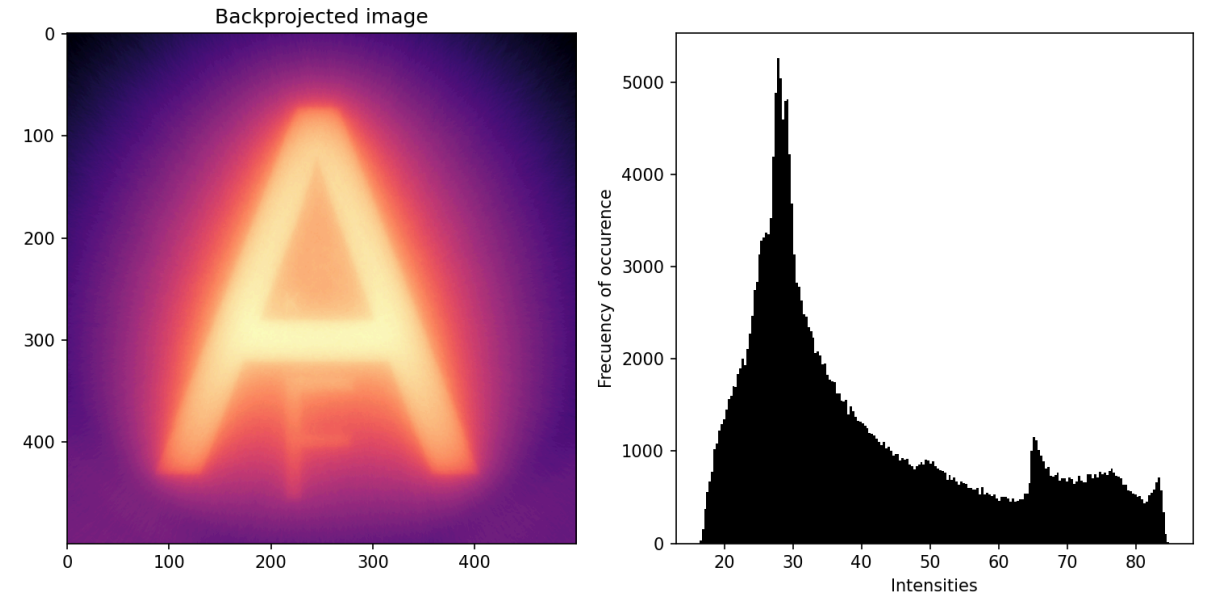
3.B. Reconstrucción tomográfica filtrada

En bloque neón hay un video que muestra en resumen cómo funciona una tomografía. Mire el video antes de continuar.

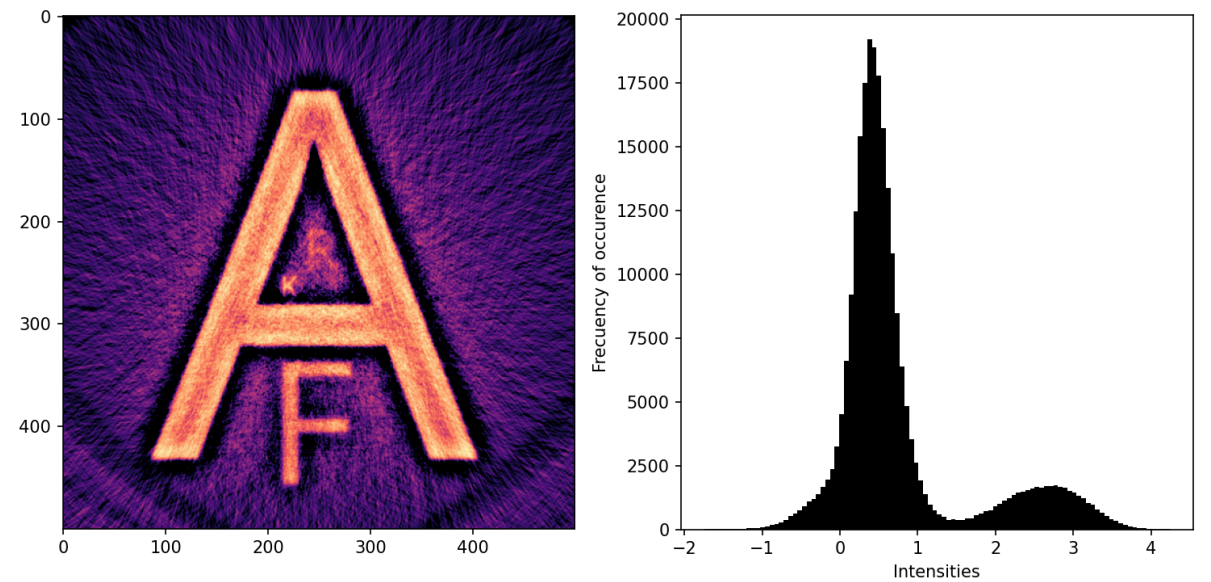
- Para cada ángulo, se toma una imagen 1D de los rayos X atravesando la parte del cuerpo que se quiere observar.
 - El resultado es la una suma de intensidades para ese ángulo, una señal 1D.
- Se hace esto para muchos ángulos y se guardan las proyecciones.
- Luego, cada imagen 1D se convierte en una imagen 2D repitiendo los datos hacia abajo.
 - La imagen resultante se rota, como se muestra en la figura



- Se suman todas estas imágenes, y así se logra una reconstrucción aproximada de la imagen original.



Pero esta imagen es muy borrosa, no se distinguen los detalles. Para mejorar los detalles, diseñe un filtro pasa altas con forma de sigmoide (`scipy.special.expit`) y aplíquelo a cada una de las proyecciones (señales 1D) antes de la suma. Esto producirá algunos artefactos en la imagen, pero mejorará el contraste:



Pista: si tiene la señal 1D en el array `signal`, para formar la imagen ya rotada puede servirse del siguiente código:

```
from scipy import ndimage as ndi
imagen_rotada = ndi.rotate(
    np.tile(signal[:,None],rows).T,
    rotation_angle,
    reshape=False,
    mode="reflect"
)
```

Donde `rotation_angle` es el ángulo de rotación en grados, `rows` es el número de filas (píxeles en y) de la imagen resultante, y los demás argumentos se encargan de conservar el brillo.

Encontrará una carpeta con muchos conjuntos de datos de proyecciones de una tomografía craneal. Use el número de su grupo. Si por algún motivo algún miembro de su grupo tienen sensibilidad a las imágenes anatómicas, use a `skelly.npy`

Usando `matplotlib`. muestre en dos subplots la imagen reconstruida sin filtro y luego con filtro. Guarde la imagen filtrada resultante en `3.png` **(15pt)** No incluya el histograma.

Datos cortesía de [Prof. Aaron J. Newman, Neural Data Science Book](#).